Day 124: The Singular Value Decomposition, and Its Relationship to PCA

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 6 | Complete study guide

Watch: Singular Value Decomposition (SVD): Overview (Steve Brunton) | Singular Value Decomposition (the SVD) (MIT OpenCourseWare)

Script: `day124_svd.py` · every number in this guide is copied by program from `day124_run_output.txt`, the real output of that script.

Every matrix factors as U S V^T. Applied to centred data, the SVD reproduces Day 123's PCA exactly without forming the covariance matrix, and that matters: computing it through A^TA, as Day 123 did, loses every singular value below about 10^{-8} of the largest (the 9th of a Hilbert matrix is off by 2.7e+01 times its own size). On a real photograph, rank 20 keeps the picture recognisable with 0.078 of the numbers, and the error is exactly the dropped singular values. With half the pixels hidden, a rank-10 fit fills them in with error 0.1223, against 0.3069 for column means; higher ranks never settle.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

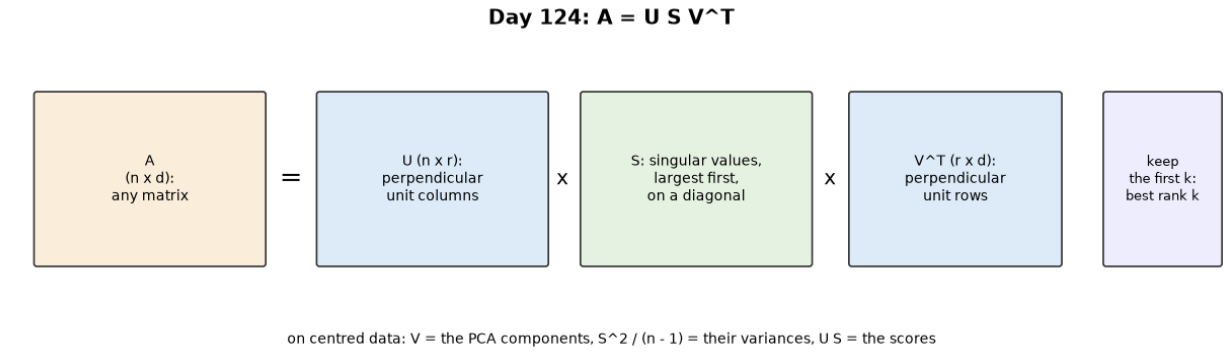
The **singular value decomposition** (SVD) writes any matrix A (n x d) as U S V^T: the columns of U and of V are perpendicular unit vectors, and S is diagonal with the **singular values**, largest first. Each triplet (u, s, v) says: A turns the direction v into s times the direction u.

Keeping the k largest gives the **best rank-k approximation** of A. On centred data the SVD is PCA: V holds the components and $S^2/(n-1)$ their variances.

REAL-WORLD SCENARIO

A streaming service's table of who watched what is mostly empty. Factoring it into a few 'taste' directions lets it guess the empty cells: this is low-rank matrix completion, demonstrated here on the pixels of a photograph, where the hidden answers are known.

SYSTEM DIAGRAM



A = U S V^T; keep the first k terms for the best rank-k approximation.

Today's part for the week's build

`day124_exercise.py` is the five SVD tools, each checked on a case with a known answer. Its real output:

ACTUAL OUTPUT

```
[[3, 0], [4, 5]]: singular values [6.7082, 2.2361]; by hand sqrt(45) = 6.7082, sqrt(5) = 2.2361
rank-5 error 16.184235; square root of the dropped singular values squared 16.184235
power iteration, top 3: [9.297865, 8.679791, 8.23819]; np.linalg.svd: [9.297865, 8.679791,
    8.23819]
exactly rank-2 matrix, half hidden: largest error on hidden entries after 76 rounds 7.6e-05
pseudoinverse against lstsq: largest gap 5.3e-16
```

Data: digits and Diabetes, and china.jpg (a photograph of a pagoda that ships with scikit-learn), turned to grey. np.linalg.svd and np.linalg.lstsq are the checks.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: a 2 x 2 matrix

DEFINITION — start here, no prior knowledge assumed

A = [[3, 0], [4, 5]]. Its singular values are the square roots of the eigenvalues of $A^T A$.

REAL-WORLD SCENARIO

A rubber sheet stretched by a machine: in one direction it is pulled most, in the perpendicular one least.

MATHEMATICS

$$A = U S V^T; \quad A^T A = V S^2 V^T; \quad A v_i = s_i u_i$$

IMPLEMENTATION

```
def svd_via_eig(A):
    vals, V = np.linalg.eigh(A.T @ A)
    o = np.argsort(vals)[::-1]; vals, V = vals[o], V[:, o]
    s = np.sqrt(np.maximum(vals, 0))
    U = (A @ V) / np.where(s > 0, s, 1)
    return U, s, V
```

ACTUAL OUTPUT

```
A^T A = [[25, 20], [20, 25]]; its eigenvalues are 25 + 20 = 45 and 25 - 20 = 5, so the singular
    values are sqrt(45) and sqrt(5)
by hand: 6.7082 and 2.2361; from A^T A here: 6.7082 and 2.2361; np.linalg.svd: [6.7082, 2.2361]
U S V^T rebuilds A: True; columns of U perpendicular: True
V's first column (0.7071, 0.7071) = (1, 1)/sqrt(2): A stretches that direction most, by 6.7082
Actual output from this lesson's run.
```

What it means

A^TA = [[25, 20], [20, 25]] has eigenvalues 45 and 5, so the singular values are 6.7082 and 2.2361, as the library agrees. The direction (1, 1)/√2 is stretched most.

WHY CHOOSE THIS?

The definition, checked by hand.

WHY NOT THIS? (TRADEOFFS)

This route squares the matrix; section 3 shows what that costs.

2. SVD is PCA

DEFINITION — start here, no prior knowledge assumed

The digits, centred. Day 123's components and eigenvalues against the SVD of the data itself.

REAL-WORLD SCENARIO

Two routes to the same summit: one climbs the covariance matrix, the other goes straight up the data.

MATHEMATICS

$$A = X - mean = U\,S\,V^T \Rightarrow A^TA/(n-1) = V\,(S^2/(n-1))\,V^T$$

ACTUAL OUTPUT

```
s^2 / (n - 1) against Day 123's eigenvalues: largest gap 3.4e-13 (largest eigenvalue 179.0)
right singular vectors against Day 123's components, first 20: smallest |cos| 1.0000000000
U S equals the component scores (X - mean) V: True
the SVD reaches PCA without forming the covariance matrix A^T A / (n - 1)
Actual output from this lesson's run.
```

What it means

The variances agree to 3.4e-13 (on a largest eigenvalue of 179.0), the components to the last digit shown, and U S equals the scores. PCA is one use of the SVD; most libraries compute PCA this way.

WHY CHOOSE THIS?

PCA without forming the covariance matrix.

WHY NOT THIS? (TRADEOFFS)

The SVD of a large data matrix is itself expensive; truncated methods compute just the top k.

3. Three routes, and what squaring costs

DEFINITION — start here, no prior knowledge assumed

The singular values by np.linalg.svd, through A^TA, and by power iteration on A and A^T in turn. Then the 12 x 12 Hilbert matrix, whose singular values span 16 orders of magnitude.

REAL-WORLD SCENARIO

Measuring a hair's width with a ruler marked in metres, after first squaring it: the answer vanishes in the rounding.

MATHEMATICS

condition number $\kappa = s_{max}/s_{min}$; $\kappa(A^TA) = \kappa(A)^2$; *double precision keeps about 16 digits*

IMPLEMENTATION

```
def svd_power(A, k, iters=3000, tol=1e-12, seed=0):
    rng = np.random.RandomState(seed); U, s, V = [], [], []
    for _ in range(k):
        v = rng.normal(size=A.shape[1]); v /= np.linalg.norm(v)
        for _ in range(iters):
            u = A @ v - sum(sj * uj * (vj @ v) for sj, uj, vj in zip(s, U, V))
            u /= np.linalg.norm(u)
            w = A.T @ u - sum(sj * vj * (uj @ u) for sj, uj, vj in zip(s, U, V))
            sigma = np.linalg.norm(w); w /= sigma
            done = np.linalg.norm(w - v) < tol
            v = w
            if done:
                break
        U.append(u); s.append(sigma); V.append(v)
    return np.array(U).T, np.array(s), np.array(V).T
```

ACTUAL OUTPUT

```
digits (well conditioned at the top): top 5 singular values, relative gap from np.linalg.svd: through A^T A
    4.0e-16, by power iteration 6.0e-16
Hilbert 12 x 12: singular values from 1.795 down to 9.93e-17; condition number 1.8e+16
    singular value   np.linalg.svd   through A^T A  relative error
                 1       1.795e+00       1.795e+00         4.9e-16
                 6       1.116e-05       1.116e-05         1.3e-07
                 9       2.252e-10       6.202e-09         2.7e+01
                11       2.650e-14       0.000e+00         1.0e+00
                12       9.931e-17       0.000e+00         1.0e+00
A^T A squares the condition number: values below about sqrt(machine precision) x the largest are lost in
    rounding
Actual output from this lesson's run.
```

What it means

On the digits all three agree (4.0e-16 and 6.0e-16). On the Hilbert matrix the A^TA route is fine for the large singular values but loses the small ones: the 6th is still good to 1.3e-07, the 9th is off by 2.7e+01 times its own size, and the 11th and 12th become 0. Squaring the matrix squares its condition number, so values below about 10^{-8} of the largest drown in rounding. That is why standard libraries take other routes.

WHY CHOOSE THIS?

Power iteration needs only products with A and A^T: good for huge sparse matrices.

WHY NOT THIS? (TRADEOFFS)

Forming A^TA is easy but throws away half the precision.

4. A photograph at rank k

DEFINITION — start here, no prior knowledge assumed

china.jpg in grey, a 427 x 640 matrix of brightness values, approximated with k singular triplets.

REAL-WORLD SCENARIO

Describing a picture to someone over the phone: a few broad strokes first, then finer and finer detail.

MATHEMATICS

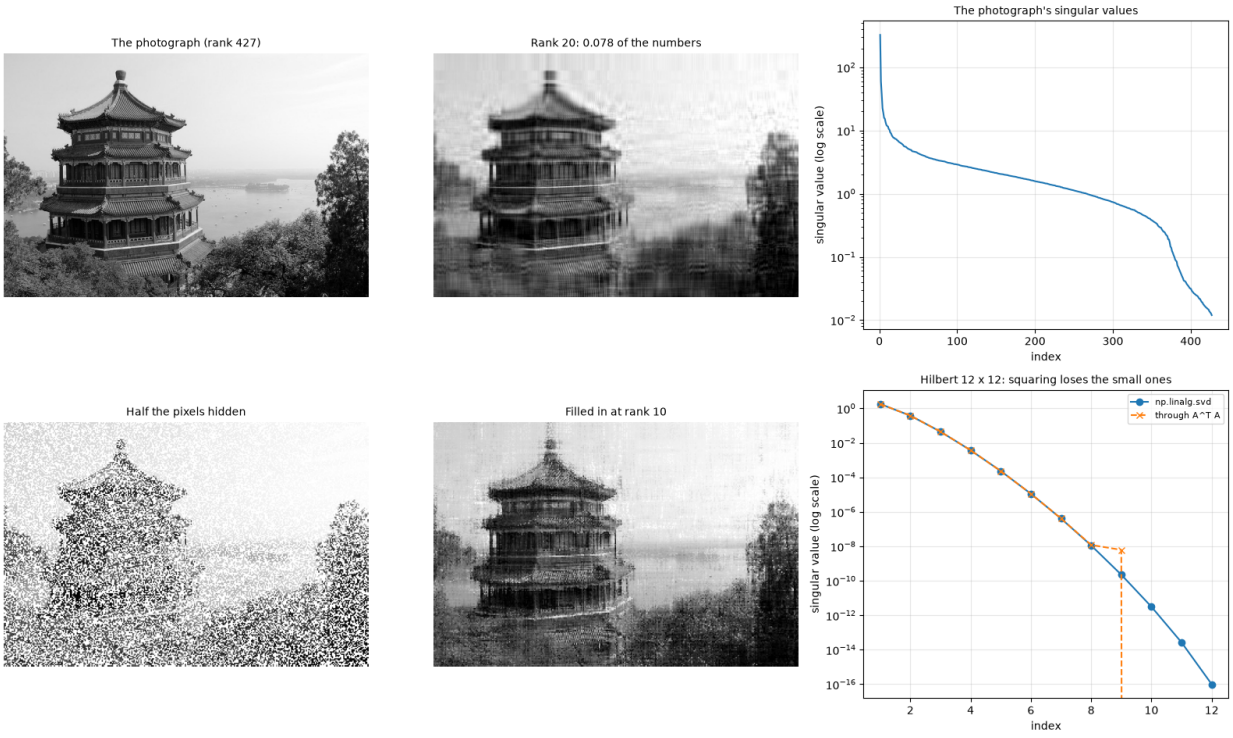
$|A - A_k| = \sqrt{(s_{k+1}{}^2 + \ldots + s_r{}^2)}$ *(Eckart–Young: no rank-k matrix does better)*

IMPLEMENTATION

```
def low_rank(U, s, V, k):
    return (U[:, :k] * s[:k]) @ V[:, :k].T
```

ACTUAL OUTPUT

```
   rank  relative error  from the dropped singular values  numbers stored  share of pixels
      1          0.2917                           0.2917           1,068            0.004
      5          0.1841                           0.1841           5,340            0.020
     10          0.1602                           0.1602          10,680            0.039
     20          0.1364                           0.1364          21,360            0.078
     50          0.1028                           0.1028          53,400            0.195
    100          0.0736                           0.0736         106,800            0.391
    200          0.0361                           0.0361         213,600            0.782
the error of the best rank-k picture is exactly the size of the singular values left out (the Eckart-Young
    theorem)
Actual output from this lesson's run.
```



Top: the photograph, its rank-20 version, and its singular values. Bottom: half the pixels hidden, the rank-10 fill, and the Hilbert matrix's singular values by two routes.

What it means

The measured error and the error predicted from the dropped singular values agree at every rank: rank 20 leaves 0.1364 of the picture's size as error while storing 0.078 of the numbers. The singular values fall steeply at first and then slowly, so the first few terms carry the broad shapes and fine texture needs hundreds.

WHY CHOOSE THIS?

The optimal low-rank summary, with an exact error.

WHY NOT THIS? (TRADEOFFS)

Image formats such as JPEG use transforms designed for photographs (not compared here); low rank suits tables more than textures.

5. Filling in missing pixels

DEFINITION — start here, no prior knowledge assumed

Half the pixels of a 214 x 320 version of the photograph are hidden at random. Repeatedly fit rank k to the current guess, keep the known pixels, and replace the unknown ones with the fit.

REAL-WORLD SCENARIO

Restoring a mosaic with half its tiles missing, from the patterns in the tiles that remain.

IMPLEMENTATION

```
def complete(M, mask, k, iters=2000, tol=1e-6):
    X = np.where(mask, M, M[mask].mean())
    for t in range(iters):
        U, s, Vt = np.linalg.svd(X, full_matrices=False)
        L = (U[:, :k] * s[:k]) @ Vt[:k]
        change = np.linalg.norm((L - X)[~mask]) / max(np.linalg.norm(X[~mask]), 1e-12)
        X = np.where(mask, M, L)
        if change < tol:
            break
    return X, t + 1
```

ACTUAL OUTPUT

```
known pixels 0.501; error on the hidden pixels (RMSE, 0 to 1 scale): filled with column means 0.3069
  rank   2: RMSE 0.1543 after 35 rounds
  rank   5: RMSE 0.1273 after 97 rounds
  rank  10: RMSE 0.1223 after 607 rounds
  rank  20: RMSE 0.1508 after 2000 rounds  (the 2,000-round limit: not converged)
  rank  40: RMSE 0.4764 after 2000 rounds  (the 2,000-round limit: not converged)
  rank  80: RMSE 0.2886 after 2000 rounds  (the 2,000-round limit: not converged)
the high ranks after only 200 rounds: rank 20 RMSE 0.1428, rank 40 RMSE 0.2481, rank 80 RMSE 0.2796: they get
    worse the longer they run
too low a rank blurs; too high a rank has the freedom to match the known pixels in ways that do not carry over
    to the hidden ones
Actual output from this lesson's run.
```

What it means

Every rank beats filling each column with its mean (0.3069). Rank 10 is best (0.1223, settling after 607 rounds). Ranks 20 and above never settled within 2,000 rounds and ended worse; after only 200 rounds they were better (rank 40: 0.2481, against 0.4764 at the end), so the longer they ran, the further they drifted. With freedom to spare, the fit matches the known pixels in ways that do not carry over to the hidden ones.

WHY CHOOSE THIS?

Real gaps filled from structure alone, the idea behind recommender systems.

WHY NOT THIS? (TRADEOFFS)

The rank must be chosen, and too high a rank gets worse with more rounds.

6. Least squares through the pseudoinverse

DEFINITION — start here, no prior knowledge assumed

Diabetes progression from 10 measurements, solved with the SVD, plain and with the smallest singular values dropped.

REAL-WORLD SCENARIO

Two witnesses who always agree cannot tell you how much each of them matters; any split of the credit fits their story.

MATHEMATICS

$x = V S^{-1} U^T b; \quad$ *truncated: keep only the k largest* s_i

IMPLEMENTATION

```
def pinv_solve(A, b, k=None):
    U, s, Vt = np.linalg.svd(A, full_matrices=False); k = len(s) if k is None else k
    return Vt[:k].T @ ((U[:, :k].T @ b) / s[:k])
```

ACTUAL OUTPUT

```
pseudoinverse against np.linalg.lstsq: largest coefficient gap 1.4e-12
singular values of the training matrix: largest 17.61, smallest 0.0793
the smallest one's direction is mostly 's1' (+0.71) and 's2' (-0.55); their correlation 0.890: one nearly
    repeats the other
   kept  largest |coefficient|  test RMSE
     11                 722.2     55.862
     10                 584.6     56.252
      9                 593.7     56.144
      8                 520.4     55.295
      6                 523.0     55.313
      4                 260.7     59.266
dropping the smallest singular values tames the coefficients; it changes the test error only a little here
Actual output from this lesson's run.
```

What it means

The pseudoinverse matches lstsq (1.4e-12). The smallest singular value (0.0793) belongs to a direction made mostly of 's1' and 's2', two blood measurements correlated at 0.890: dividing by it inflates the coefficients to 722.2. Dropping it and a few more shrinks them (520.4 with 8 kept) while the test error barely moves (55.862 to 55.295); dropping too many hurts (59.266 with 4).

WHY CHOOSE THIS?

Stable coefficients when features repeat each other.

WHY NOT THIS? (TRADEOFFS)

Which directions to drop is chosen by size, not by usefulness to the target.

Interview preparation — technical questions

Q: What is the SVD?

A: $A = U S V^T$: perpendicular input and output directions, and how much A stretches each.

Q: How is PCA computed in practice?

A: By the SVD of the centred data, not the covariance: same answer (3.4e-13), better precision.

Q: Why not compute the SVD from A^TA?

A: It squares the condition number: the Hilbert matrix's 9th singular value was off by 27 times its own size.

Q: How would you compress a matrix?

A: Keep the top k singular triplets; the error is exactly the dropped singular values.

Q: How do recommender systems fill gaps?

A: Low-rank completion: here rank 10 filled hidden pixels at 0.1223 against 0.3069 for column means.

Interview preparation — theoretical questions

Q: What does the Eckart–Young theorem say?

A: The truncated SVD is the best rank-k approximation; the measured and predicted errors matched at every rank.

Q: What is the pseudoinverse?

A: $V S^{-1} U^T$: the least-squares solution, and the smallest-norm one when there are many.

Q: Why do tiny singular values cause trouble?

A: Dividing by them inflates coefficients along directions the data barely supports (722.2 here).

Q: How does power iteration find singular vectors?

A: Multiply by A, then by A^T, renormalise, repeat; remove each triplet found.

Key takeaway and what's next

> **KEY TAKEAWAY**
> The SVD factors any matrix into directions in, directions out, and stretches between them. It is how PCA should be computed, the optimal way to keep a matrix's main structure, and a way to fill in what is missing. Its numbers come with guarantees, and its one classic trap, squaring the matrix, was measured rather than assumed.

- SVD = PCA: variances agree to 3.4e-13.
- Through A^TA: the Hilbert matrix's small singular values lost (the 11th and 12th became 0).
- Photograph: rank 20, 0.078 of the numbers, error 0.1364, exactly the dropped singular values.
- Half the pixels hidden: rank 10 best (0.1223); column means 0.3069.

Suggested watch order, if starting from zero

- 1. Singular Value Decomposition (SVD): Overview (Steve Brunton) — the SVD's main ideas, from the factorisation to truncation.
- 2. Singular Value Decomposition (the SVD) (MIT OpenCourseWare) — an MIT lecture on the SVD.

Day 125 preview

- **The Week 18 build**: t-SNE and UMAP from scratch, compared on real high-dimensional data, with the week's clustering parts.